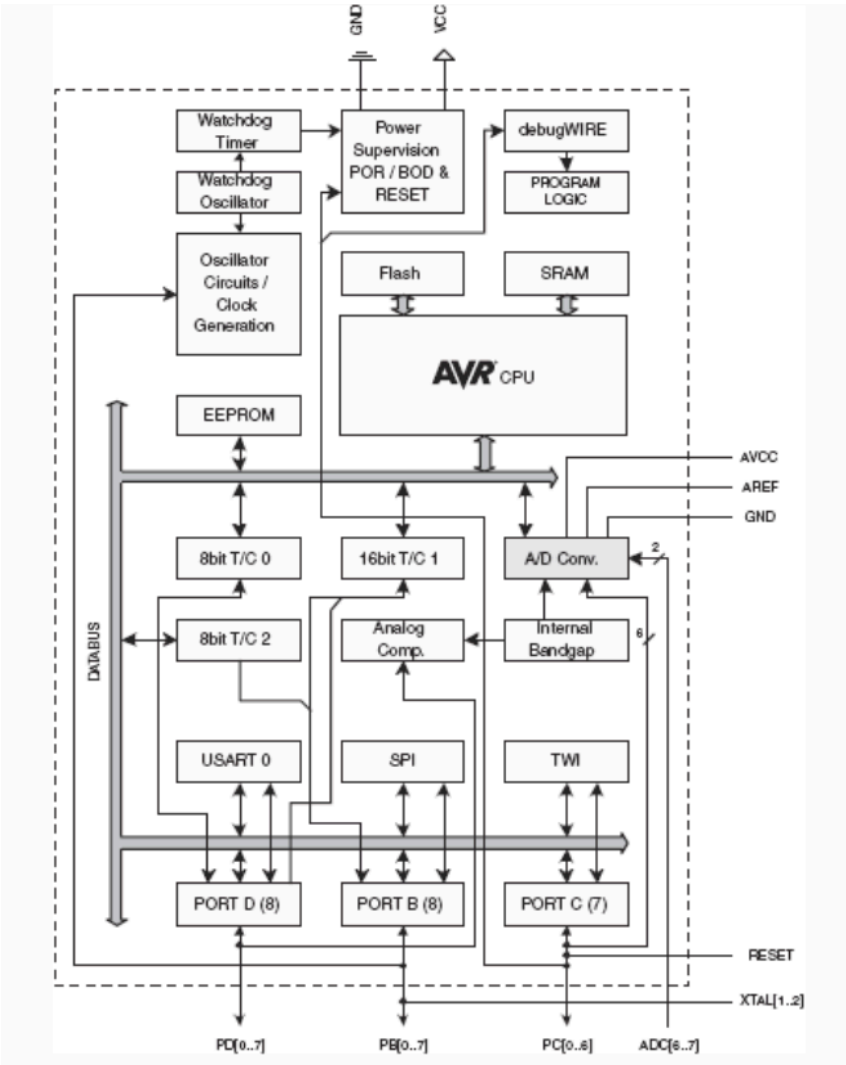
ECE 3218 Lab #3 - Parallel I/O: Lights and LED display

Background

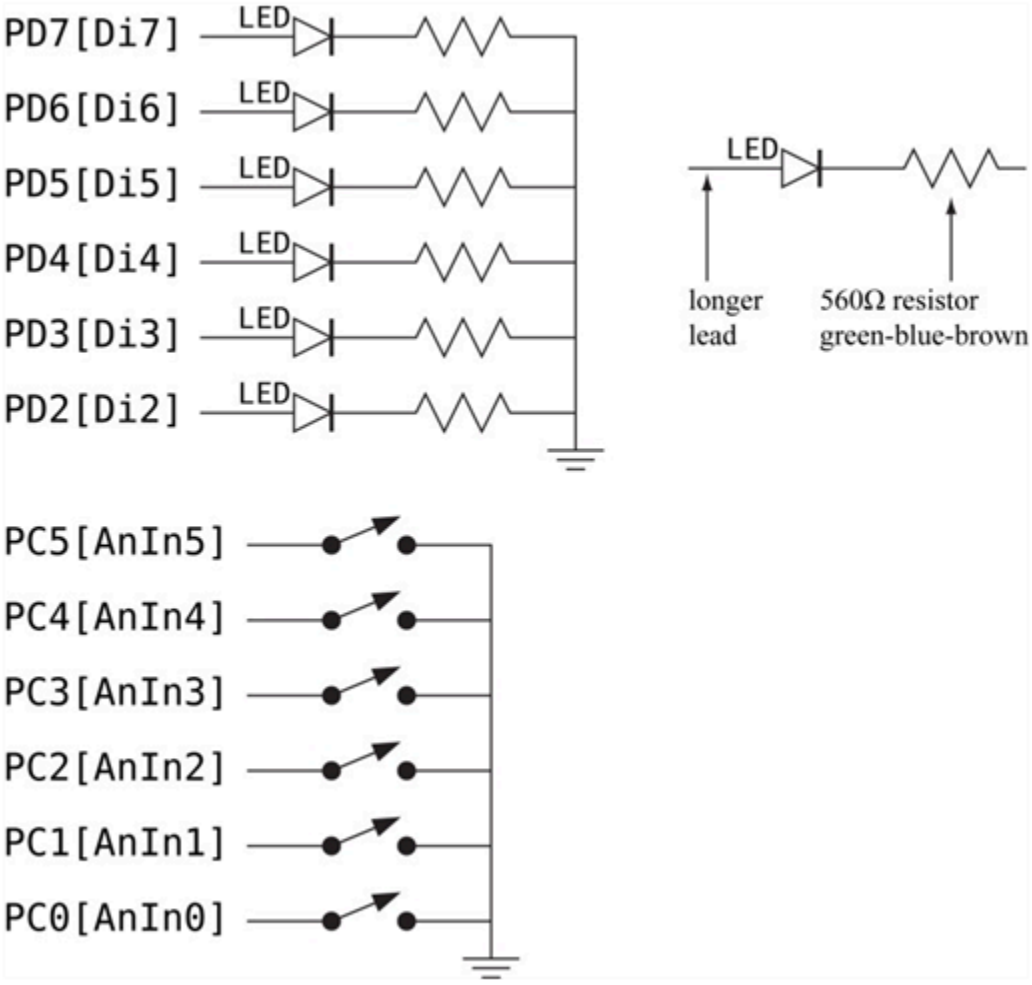
The most commonly used method of exchanging information between a microprocessor system and the outside world is through parallel I/O. The AVR microcontroller has a number of parallel ports (`B`, `C`, `D`) – `B` and `D` have 8 bits, and `C` has 7 bits. These ports have dual functions so they can be used as I/O ports or as special function pins for serial ports, or timer devices, etc.

On the Arduino board only the following bits can be used: `PORTD0-PORTD7`, `PORTC0-PORTC5`, and `PORTB0-PORTB5`.



We are going to use the breadboard in the lab to construct a simple circuit that uses switches and LEDs. The circuit diagram is shown below. `PDx` means `PORTD` bit `x`, [`Dix`] means Digital `x` (this is printed on the Arduino), `PCx` means `PORTC` bit `x`, and [`AnInx`] means Analog In `x` (this is also printed in the Arduino). `PORTC` will be used as input, and will be connected to a DIP switch. When the switch is closed, it will connect the appropriate `PORTC` bit to ground.

You will build this circuit in the lab on the breadboard. You need to locate the correct headers on the Arduino board, place the LEDs and the switch on the breadboard, and connect them using the cables provided. Be very careful with the cables: their ends are very nimble.



Be aware that LEDs are polarized: they will work only as shown on the picture above: instead of using individual components, we will be using LED and resistor arrays. Your TA will demonstrate how to connect them. When the AVR sends a logic High on a bit, the LED will light up, when it sends a logic Low, the LED will be dark. Recall that you need to set `PORTD` bits `PD7-PD2` to `OUTPUT` direction using the `DDRD` (Data Direction Register for `PORTD`). For details refer to Atmega328P datasheet section 13.2.1.

The switches can be read using `PINC` (Pins on `PORTC`). Recall that you need to set the `PORTC` bits `PC0-PC5` to `INPUT` direction using the `DDRC` (Data Direction Register for `PORTC`). You will also need to enable the 'pull-up' resistors of `PORTC` (writing 1-s into the `PORTC` bits after setting the direction) – this is needed so the signals will be correctly read is logic High when the switch is open.

Note: We read from `PC0-PC5` but write to `PD2-PD7` (6 bits in, six bits out, in parallel). Hence, the programs below will have to shift/rotate data as needed.

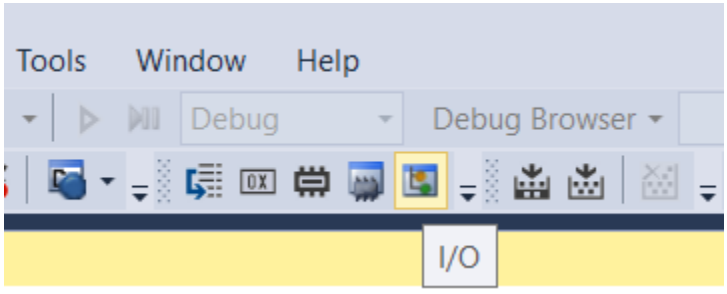
Before the Lab

Write programs that:

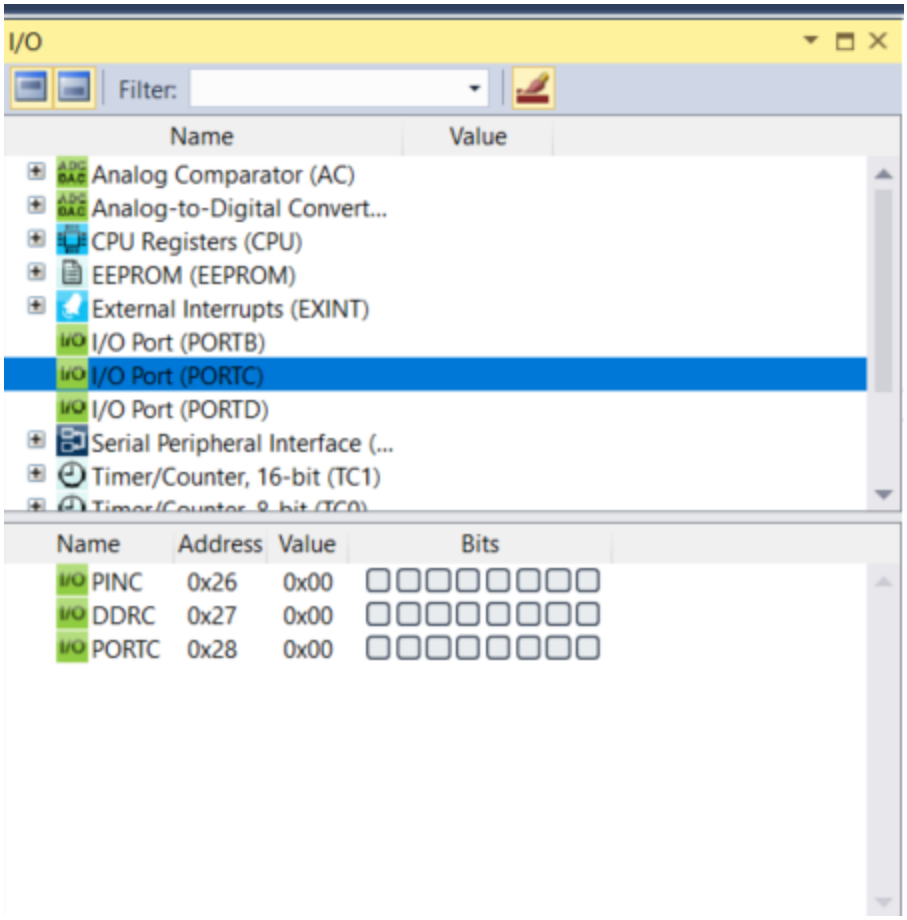
1. Copy the switch settings to the LED's. This should be a continuous process, so that when you change a switch status, the result is immediately apparent in the LEDs.
2. Rotate a single lit LED from left to right on the breadboard. Each LED should be lit for a reasonable length of times (200 ms). When the light reaches the boundary of the 8 LEDs, flash all of them then continue. Note that you'll need a routine to generate a delay.
3. Extend the previous program so that the direction of rotation depends on one of the settings of the DIP-switches.
4. Rotate a pattern obtained from switches, from right to left.

Test written programs with the Simulator

Microchip Studio provides a tool called I/O view which is able to display and edit values of the registers on the simulated microcontroller. To open this I/O view, click on the button on the toolbar showed in the picture below:



The following window appears:



In the top section of the window in the group of External Interrupts the I/O Port (PORTC) is selected. In the bottom section of this window you can see registers which belong to the PORTC. By clicking on the light grey squares in the field Bits you can set any desired value of the registers PORTC (simulate switches) during execution of your program and check if register's values on the PORTD (simulated response of LEDs) correspond with values in PORTC.

In Your Notebook

Sketch the circuit including the Arduino, switches, LEDs and resistors. Why are the resistors included?

Write out your algorithm for rotating the LEDs.

Show the delay loop you're planning to use and calculate the necessary values to achieve the desired delay.

In the Lab

Build the circuit described earlier on the breadboard. Before powering it up, have the TA verify your connections.

Assemble and debug your programs.

In program 2, decrease the delay to the point where you cannot see the individual LEDs lighting up (i.e. the LEDs are switched so fast, that your eye cannot see the difference).

After the Lab

Generate correct and annotated listings of your programs.

What is the delay time when the LEDs cannot be seen as turning on and off? Calculate the value from your delay loop.